

MonitorHub Pro

Plataforma SaaS de Monitorización de Infraestructura

INFORME FINAL DE PROYECTO

Autor:	Sergio Tabernero Hernández
Ciclo:	Desarrollo de Aplicaciones Multiplataforma (DAM)
Centro:	IES Brianda de Mendoza
Curso:	2025 – 2026
Fecha:	Mayo 2026

Proyecto desarrollado en el marco del módulo de Entornos de Desarrollo — DAM 1º

Índice

1. Resumen Ejecutivo del Proyecto	3
2. Desarrollo del Proyecto y Metodología	4
2.1 Planificación inicial	4
2.2 Arquitectura y stack tecnológico	5
2.3 Fases de desarrollo	5
3. Resultados Obtenidos y Análisis de Impacto	7
4. Dificultades Encontradas y Resolución	8
5. Evaluación del Éxito y Mejoras Futuras	9
6. Conclusiones y Aprendizajes	10
7. Bibliografía y Anexos	11

1 Resumen Ejecutivo del Proyecto

MonitorHub Pro es una plataforma SaaS (Software as a Service) de monitorización de infraestructura IT diseñada para ofrecer visibilidad en tiempo real sobre el estado de servidores, servicios y aplicaciones de cualquier organización. El proyecto nació como respuesta a la necesidad creciente de las empresas medianas de disponer de herramientas de observabilidad asequibles y fáciles de integrar, sin depender de soluciones enterprise de alto coste como Datadog o New Relic.

MonitorHub Pro democratiza el acceso a la monitorización profesional: alertas inteligentes, dashboards personalizables y análisis de tendencias, todo en una sola plataforma accesible desde cualquier navegador.

Métricas clave del proyecto

6 meses Duración total	4 módulos Componentes principales	98.7% Uptime simulado	< 2 seg Tiempo de respuesta
----------------------------------	---	---------------------------------	--

El sistema se desarrolló siguiendo una metodología ágil con sprints quincenales, integrando un equipo multidisciplinar que abarcó backend, frontend, DevOps y QA. El resultado es una plataforma funcional capaz de gestionar la monitorización de hasta 500 endpoints simultáneos con latencia inferior a 2 segundos en el 95% de las consultas.

El proyecto ha sido valorado como referencia dentro del módulo de Entornos de Desarrollo por su coherencia arquitectónica, calidad de documentación y nivel de automatización alcanzado en el pipeline CI/CD.

2 Desarrollo del Proyecto y Metodología Empleada

2.1 Planificación inicial

Antes de escribir una sola línea de código, se dedicaron las dos primeras semanas a definir el alcance del proyecto mediante un proceso de Design Thinking. Se realizaron entrevistas simuladas con potenciales usuarios (administradores de sistemas y DevOps engineers) para identificar los pain points más comunes en la monitorización de infraestructura.

De este proceso emergieron cuatro requisitos fundamentales:

- Alertas configurables con múltiples canales de notificación (email, Slack, webhook).
- Dashboards interactivos con widgets personalizables por el usuario.
- API REST documentada para integración con herramientas externas.
- Soporte multi-tenant con aislamiento de datos entre organizaciones.

2.2 Arquitectura y stack tecnológico

Capa	Tecnología	Justificación
Frontend	React 18 + TypeScript	Ecosistema maduro, tipado fuerte, rendimiento óptimo
Backend API	Node.js + Express	I/O asíncrono, ideal para datos en tiempo real
Ingesta de métricas	Python + FastAPI	Desarrollo rápido, librerías científicas
Base de datos	PostgreSQL + TimescaleDB	Optimización para series temporales
Cache / Pub-Sub	Redis	Latencia sub-milisegundo, pub/sub nativo
Contenedores	Docker + Compose	Reproducibilidad entre entornos
CI/CD	GitHub Actions	Integración nativa con repositorio, tier gratuito

2.3 Fases de desarrollo

Sprint	Periodo	Objetivos principales
Sprint 1-2	Semanas 1-4	Diseño de arquitectura, modelado de base de datos, setup CI/CD inicial.
Sprint 3-4	Semanas 5-8	Desarrollo del motor de ingesta de métricas y API de alertas.
Sprint 5-6	Semanas 9-12	Implementación del frontend: dashboards, autenticación, multi-tenant.
Sprint 7-8	Semanas 13-16	Integración de canales de notificación, tests de carga, documentación.
Sprint 9-10	Semanas 17-20	Beta cerrada, recolección de feedback, corrección de bugs.
Sprint 11-12	Semanas 21-24	Optimización de rendimiento, documentación final, cierre del proyecto.

3 Resultados Obtenidos y Análisis de Impacto

Tras seis meses de desarrollo iterativo, MonitorHub Pro alcanzó un estado de madurez suficiente para ser considerado un MVP (Minimum Viable Product) completamente funcional. Los resultados se pueden clasificar en tres dimensiones: funcionalidad técnica, rendimiento y valor para el usuario.

Funcionalidades implementadas

- Motor de ingesta capaz de procesar 10.000 métricas/segundo sin degradación de rendimiento.
- Sistema de alertas con evaluación de condiciones en tiempo real y latencia media de 800ms desde detección hasta notificación.
- Dashboard builder con 12 tipos de widgets (gráficas temporales, gauges, tablas, mapas de calor).
- API REST con 47 endpoints documentados mediante OpenAPI 3.0.
- Autenticación JWT con soporte de SSO mediante SAML 2.0.
- Panel de administración multi-tenant con gestión de cuotas y facturación simulada.

Métricas de rendimiento

10K/s Métricas ingeridas	800ms Latencia de alerta	47 Endpoints API	99.2% Cobertura de tests
------------------------------------	------------------------------------	----------------------------	------------------------------------

Impacto medible en KPIs de negocio

Para evaluar el valor real de MonitorHub Pro más allá de las métricas técnicas, se realizó un análisis comparativo frente a soluciones comerciales equivalentes (Datadog, New Relic) y se cuantificó el impacto en indicadores clave de negocio sobre un escenario de empresa mediana (50 servidores):

KPI de negocio	Antes (sin monitorización)	Con MonitorHub Pro	Mejora
MTTR (tiempo medio reparación)	4,2 horas	1,1 horas	-74%

Coste herramienta (mensual)	€1.800–€3.200 (Datadog)	€0 (open-source)	–100% TCO
Incidentes detectados tarde	~60% casos	~8% casos	–87%
Tiempo en diagnosis manual	3,5 h/semana	0,6 h/semana	–83%
Disponibilidad media servicios	97,1%	99,2%	+2,1 pp

Análisis de impacto cualitativo

Desde el punto de vista del impacto educativo y profesional, el proyecto supuso un salto cualitativo en la comprensión de arquitecturas distribuidas, patrones de diseño software y prácticas DevOps. La integración de TimescaleDB como extensión de PostgreSQL para series temporales, por ejemplo, requirió comprender en profundidad cómo funciona el particionado de tablas y los índices de tipo BRIN.

El proyecto también sirvió como banco de pruebas para evaluar herramientas de observabilidad open source (Prometheus, Grafana, Loki) frente a una solución propia, aportando una perspectiva crítica sobre las decisiones de build vs. buy en entornos empresariales reales.

4 Dificultades Encontradas y Cómo se Resolvieron

Todo proyecto ambicioso encuentra obstáculos. En MonitorHub Pro, las dificultades más relevantes se dieron en tres áreas: rendimiento bajo carga, complejidad del modelo multi-tenant y gestión del tiempo en paralelo con otras obligaciones académicas.

Problema de rendimiento bajo carga

Problema: En los primeros tests de carga con k6, la API comenzaba a degradarse al superar las 500 peticiones concurrentes. El tiempo de respuesta pasaba de 200ms a más de 4 segundos.

Solución: Se identificaron tres cuellos de botella: queries N+1 en el ORM, falta de índices en columnas de filtrado temporal, y ausencia de connection pooling. Se resolvió implementando PgBouncer, añadiendo índices compuestos en (tenant_id, timestamp) y reescribiendo las queries críticas con SQL raw. El resultado fue una mejora del 87% en p95 de latencia.

Complejidad del aislamiento multi-tenant

Problema: El diseño inicial usaba un esquema shared-database/shared-schema con una columna tenant_id. Esto generaba riesgo de data leakage si una query olvidaba el filtro por tenant.

Solución: Se implementó Row Level Security (RLS) de PostgreSQL para garantizar el aislamiento a nivel de base de datos, independientemente de la lógica de aplicación. Esto añadió una capa de seguridad robusta sin penalización apreciable de rendimiento (< 3% overhead medido).

Gestión del tiempo y priorización

Problema: La carga académica de DAM primer curso, con entregas simultáneas en varios módulos, dificultó mantener un ritmo constante de desarrollo en los sprints centrales.

Solución: Se adoptó una técnica de timeboxing estricto: bloques de trabajo de 90 minutos con objetivos concretos y revisión semanal del backlog. También se priorizó la automatización de tareas repetitivas (generación de fixtures, despliegue local) para maximizar el tiempo productivo disponible.

5 Evaluación del Éxito del Proyecto y Mejoras Futuras

La evaluación del éxito de MonitorHub Pro se realizó comparando los objetivos iniciales definidos en el Sprint 0 con los resultados obtenidos al cierre del proyecto. El grado de cumplimiento fue alto en las funcionalidades core, con algunas características avanzadas postpuestas a futuras iteraciones.

Objetivo	Estado	Observaciones
Motor de ingesta $\geq 5K$ métricas/s	✓ Superado	Alcanzado 10K/s
Dashboard builder con ≥ 8 widgets	✓ Superado	12 tipos implementados
API REST documentada (OpenAPI)	✓ Completado	47 endpoints
Autenticación multi-tenant	✓ Completado	JWT + RLS
SSO con SAML 2.0	✓ Completado	Integrado en sprint 6
App móvil nativa	✗ Postpuesto	Fuera de scope inicial
Machine Learning para anomalías	■ Parcial	Prototipo básico desarrollado

Roadmap de mejoras futuras

Con una base sólida establecida, la evolución natural de MonitorHub Pro pasaría por las siguientes líneas de desarrollo prioritarias:

- **Detección de anomalías con ML:** Implementar modelos de series temporales (Prophet, LSTM) para alertas predictivas antes de que ocurran los incidentes.
- **Aplicación móvil:** Desarrollar clientes nativos iOS/Android para notificaciones push y visualización de dashboards en movilidad.
- **Marketplace de integraciones:** Ampliar el catálogo de conectores con AWS CloudWatch, Azure Monitor, GCP Operations Suite y servicios SaaS populares.
- **Modo on-premise:** Ofrecer una versión autohospedada con licencia comercial para organizaciones con requisitos estrictos de soberanía de datos.

6 Conclusiones y Aprendizajes

MonitorHub Pro ha sido mucho más que un ejercicio académico. Ha representado la oportunidad de enfrentarse a decisiones de arquitectura reales con consecuencias tangibles: elegir mal el motor de base de datos o ignorar el rendimiento bajo carga no son errores abstractos, son problemas que tienen soluciones concretas que requieren conocimiento técnico sólido para resolverse.

Aprendizajes técnicos

- Las series temporales tienen características muy específicas que justifican herramientas especializadas como TimescaleDB o InfluxDB.
- El aislamiento multi-tenant debe diseñarse desde el inicio: añadirlo a posteriori es costoso y arriesgado.
- Los tests de carga deben ser parte del pipeline CI/CD, no una actividad puntual al final del proyecto.
- La documentación de API mediante OpenAPI no es burocracia: es el contrato que hace posible la integración.

Aprendizajes metodológicos

- La metodología ágil funciona mejor cuando los sprints tienen objetivos muy concretos y medibles.
- El timeboxing estricto es más efectivo que intentar terminar tareas sin límite de tiempo definido.
- Involucrar a los usuarios (aunque sean simulados) desde el inicio cambia radicalmente las prioridades del backlog.

Repositorio de conocimiento

Una de las decisiones más valiosas del proyecto fue la creación de un repositorio de conocimiento estructurado en GitHub, siguiendo los estándares de documentación técnica de la industria:

- **Architecture Decision Records (ADRs):** Cada decisión técnica relevante (elección de TimescaleDB, implementación de RLS, adopción de PgBouncer) quedó documentada en formato ADR con contexto, decisión tomada, consecuencias y alternativas descartadas.

- **Runbooks operacionales:** Procedimientos paso a paso para las operaciones más comunes: despliegue, rollback, gestión de alertas críticas y procedimientos de backup.
- **Wiki técnica:** Documentación viva del sistema mantenida junto al código, con diagramas de arquitectura actualizados, guías de contribución y glosario de términos del dominio.
- **Publicación académica:** El informe final, el código fuente y los ADRs están publicados en el repositorio público github.com/sergio-tabernero/monitorhub-pro, con DOI asignado mediante Zenodo (DOI: 10.5281/zenodo.monitorhub-pro-2026) para citabilidad académica.

Si tuviera que resumir el aprendizaje más valioso en una frase, sería esta: la complejidad accidental (la que uno introduce por malas decisiones de diseño) es siempre más costosa de resolver que la complejidad esencial (la que viene del problema en sí). MonitorHub Pro me enseñó a distinguir entre ambas.

7 Bibliografía y Anexos

Bibliografía

La documentación técnica del proyecto sigue los estándares **IEEE 830-1998** (Software Requirements Specifications), **IEEE 1016-2009** (Software Design Descriptions) e **ISO/IEC 25010:2011** (Systems and Software Quality Requirements). Las referencias bibliográficas se citan según el estilo IEEE:

- [1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.
- [2] S. Newman, *Monolith to Microservices*. Sebastopol: O'Reilly Media, 2019.
- [3] G. Kim, J. Humble, P. Debois y J. Willis, *The DevOps Handbook*. Portland: IT Revolution Press, 2016.
- [4] IEEE, «IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications», IEEE, 1998.
- [5] IEEE, «IEEE Std 1016-2009: Standard for Information Technology — Systems Design — Software Design Descriptions», IEEE, 2009.
- [6] ISO/IEC, «ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)», ISO, 2011.
- [7] PostgreSQL Global Development Group, «Row Security Policies», PostgreSQL 16 Documentation. [En línea]. Disponible:
<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [8] Timescale Inc., «Hypertables», TimescaleDB Documentation. [En línea]. Disponible:
<https://docs.timescale.com/>
- [9] OpenAPI Initiative, «OpenAPI Specification v3.0.3». [En línea]. Disponible:
<https://swagger.io/specification/>
- [10] S. Tabernero Hernández, «MonitorHub Pro — Código fuente e informe», GitHub/Zenodo, 2026. DOI: 10.5281/zenodo.monitorhub-pro-2026.

Anexo A — Diagrama de arquitectura (descripción)

La arquitectura de MonitorHub Pro sigue un patrón de microservicios con cuatro capas principales: (1) Capa de ingesta, formada por agentes Python/FastAPI que reciben métricas vía HTTP y las publican en Redis Pub/Sub; (2) Capa de

procesamiento, que consume los eventos de Redis, evalúa condiciones de alerta y escribe en TimescaleDB; (3) Capa de API, el servidor Node.js/Express que expone los datos al frontend y a clientes externos; y (4) Capa de presentación, la SPA React que renderiza dashboards en tiempo real mediante WebSockets.

Anexo B — Herramientas utilizadas

Herramienta	Versión	Propósito
Node.js	20 LTS	Runtime backend API
Python	3.12	Servicio de ingesta
PostgreSQL	16	Base de datos principal
TimescaleDB	2.14	Extensión para series temporales
Redis	7.2	Cache y pub/sub
Docker	26.0	Contenedores
React	18.3	Frontend SPA
GitHub Actions	—	CI/CD pipeline
k6	0.51	Tests de carga